Formatting Ranges

Document #:	P2286R0
Date:	2021-01-14
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Introduction

[LWG3478] addresses the issue of what happens when you split a string and the last character in the string is the delimiter that you are splitting on. One of the things I wanted to look at in research in that issue is: what do *other* languages do here?

For most languages, this is a pretty easy proposition. Do the split, print the results. This is usually only a few lines of code.

Python:

```
print("xyx".split("x"))
```

outputs

```
['', 'y', '']
```

Java (where the obvious thing prints something useless, but there's a non-obvious thing that is useful):

```
import java.util.Arrays;
```

```
class Main {
  public static void main(String args[]) {
    System.out.println("xyx".split("x"));
    System.out.println(Arrays.toString("xyx".split("x")));
  }
}
```

outputs

```
[Ljava.lang.String;@76ed5528
[, y]
```

Rust (a couple options, including also another false friend):

```
use itertools::Itertools;

fn main() {
    println!("{:?}", "xyx".split('x'));
    println!("[{}]", "xyx".split('x').format(", "));
    println!("{:?}", "xyx".split('x').collect::<Vec<_>>());
}
```

outputs

```
Split(SplitInternal { start: 0, end: 3, matcher: CharSearcher { haystack:
        "xyx", finger: 0, finger_back: 3, needle: 'x', utf8_size: 1,
        utf8_encoded: [120, 0, 0, 0] }, allow_trailing_empty: true,
        finished: false })
[, y, ]
["", "y", ""]
```

Kotlin:

```
fun main() {
    println("xyx".split("x"));
}
```

outputs

```
[, y, ]
```

Go:

```
package main
import "fmt"
import "strings"

func main() {
    fmt.Println(strings.Split("xyx", "x"));
}
```

outputs

```
[ y ]
```

JavaScript:

```
console.log('xyx'.split('x'))
```

outputs

```
[ '', 'y', '' ]
```

And so on and so forth. What we see across these languages is that printing the result of split is pretty easy. In most cases, whatever the print mechanism is just works and does something meaningful. In other cases, printing gave me something other than what I wanted but some other easy, provided mechanism for doing so.

Now let's consider C++.

```
#include <iostream>
#include <string>
#include <ranges>
#include <format>

int main() {
    // need to predeclare this because we can't split an rvalue string
    std::string s = "xyx";
    auto parts = s | std::views::split('x');

    // nope
    std::cout << parts;

    // nope (assuming std::print from P2093)
    std::print("{}", parts);


    std::cout << "[";
    char const* delim = "";
    for (auto part : parts) {
        std::cout << delim;

        // still nope
        std::cout << part;

        // also nope
        std::print("{}", part);

        // this finally works
        std::ranges::copy(part, std::ostream_iterator<char>(std::cout));

        // as does this
        for (char c : part) {
            std::cout << c;
        }
        delim = ", ";
    }
    std::cout << "]\n";
}
```

This took me more time to write than any of the solutions in any of the other languages. Including the Go solution, which contains 100% of all the lines of Go I've written in my life.

Printing is a fairly fundamental and universal mechanism to see what's going on in your program. In the context of ranges, it's probably the most useful way to see and understand what the various range adapters actually do. But none of these things provides an `operator<<` (for `std::cout`) or a formatter specialization (for `format`). And the further problem is that as a user, I can't even do anything about this. I can't just provide an `operator<<` in `namespace std` or a very broad specialization of `formatter` - none of these are program-defined types, so it's just asking for clashes once you start dealing with bigger programs.

The only mechanisms I have at my disposal to print something like this is either

1. nested loops with hand-written delimiter handling (which are tedious and a bad solution), or
2. at least replace the inner-most loop with a `ranges::copy` into an output iterator (which is more differently bad), or
3. Write my own formatting library that I *am* allowed to specialize (which is not only bad but also ridiculous)
4. Use `fmt::format`.

1.1 Implementation Experience

That's right, there's a third option for C++ that I haven't shown yet, and that's this:

```
#include <ranges>
#include <string>
#include <fmt/ranges.h>

int main() {
    std::string s = "xyx";
    auto parts = s | std::views::split('x');

    fmt::print("{}\n", parts);
    fmt::print("[{}]\n", fmt::join(parts, ","));
}
```

outputting

```
{{}, {'y'}}
[{},{'y'}]
```

And this is great! It's a single, easy line of code to just print arbitrary ranges (include ranges of ranges).

And, if I want to do something more involved, there's also `fmt::join`, which lets me specify both a format specifier and a delimiter. For instance:

```
std::vector<uint8_t> mac = {0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};
fmt::print("{:02x}\n", fmt::join(mac, ":"));
```

outputs

```
aa:bb:cc:dd:ee:ff
```

`fmt::format` (and `fmt::print`) solves my problem completely. `std::format` does not, and it should.

2 Proposal Considerations

The Ranges Plan for C++23 [P2214R0] listed as one of its top priorities for C++23 as the ability to format all views. Let's go through the issues we need to address in order to get this functionality.

2.1 What types to print?

The standard library is the only library that can provide formatting support for standard library types and other broad classes of types like ranges. In addition to ranges (both the conrete containers like `vector<T>` and the range adaptors like `views::split`), there are several very commonly used types that are currently not printable.

The most common and important such types are `pair` and `tuple` (which ties back into Ranges even more closely once we adopt `views::zip` and `views::enumerate`). `fmt` currently supports printing such types as well:

```
fmt::print("{}\n", std::pair(1, 2));
```

outputs

```
(1, 2)
```

Another common and important set of types are `std::optional<T>` and `std::variant<Ts...>`. `fmt` does not support printing any of the sum types. There is not an obvious representation for them in C++ as there might be in other languages (e.g. in Rust, an `Option<i32>` prints as either `Some(42)` or `None`, which is also the same syntax used to construct them).

However, the point here isn't necessarily to produce the best possible representation (users who have very specific formatting needs will need to write custom code anyway), but rather to provide something useful. And it'd be useful to print these types as well. However, given that `optional` and `variant` are both less closely related to Ranges than `pair` and `tuple` and also have less obvious representation, they are less important.

2.2 What representation?

Should `vector<int>{1, 2, 3}` be printed as `{1, 2, 3}` (as `fmt` currently does and as the type is constructed in C++) or `[1, 2, 3]` (as is typical representationally outside of C++)?

Should `pair<int, char>{3, 'x'}` be printed as `{3, 'x'}` (as the type is constructed) or as `(3, 'x')` (as `fmt` currently does and is typical representationally outside of C++)?

I think the specific choice of formatting result matters a lot less than the fact that there is a formatting result at all, and would be happy to leave it up to the implementation.

Following that, I would prefer if a range was represented in a way that was identifiably distinct from a tuple.

§ 2.3 What additional functionality?

There's three layers of potential functionality:

1. Top-level printing of ranges: this is `fmt::print("{}", r);`
2. A format-joiner which allows providing a format specifier for each element and a delimiter: this is `fmt::print("{:02x}", fmt::join(r, ":"))`.
3. A more involved version of a format-joiner which takes a delimiter and a callback that gets invoked on each element. `fmt` does not provide such a mechanism, though the Rust itertools library does:

```
let matrix = [[1., 2., 3.],
              [4., 5., 6.]];
let matrix_formatter = matrix.iter().format_with("\n", |row, f| {
                                    f(&row.iter().format_with(", ", |elt, g|
            g(&elt)))
                                });
assert_eq!(format!("{}", matrix_formatter),
               "1, 2, 3\n4, 5, 6");
```

This paper suggests the first two and encourages research into the third.

§ 2.4 How to support those views which are not `const`-iterable?

There are several C++20 range adapters which are not `const`-iterable. These include `views::filter` and `views::drop_while`. But `std::format` takes its arguments by `Args const&...`, so how could they be printable?

`fmt` handles this just fine even with its formatter specialization taking by const by converting the range to a view first.

So we can do something like:

```
template <range R, typename Char>
struct formatter<R, Char>
{
    template <typename FormatContext>
    auto format(R const& rng, FormatContext& ctx) {
        auto values = std::views::all(rng);
        // ...
    }
};
```

But, this still doesn't cover all the cases. If `R` is a type such that `R const` is a `view` but `R` is move-only, that won't compile. We can work around this case. But if `R` is a type that such that `view<R> and not range<R const> and not copyable<R>`, there's really no way of getting this to work without changing the API.

If we do want to support this case (and `fmt` does not), then we will need to change the API of `format` to take by forwarding reference. Notably, the proposed `std::generator<T>` is one such type [P2168R0]. The workaround here is to use `fmt::join` instead (which does take by forwarding reference).

Do we care about supporting this use-case directly?

```
auto ints_coro(int n) -> std::generator<int> {
    for (int i = 0; i < n; ++i) {
        co_yield i;
    }
}

fmt::print("{}", ints_coro(10)); // error
fmt::print("[{}]", fmt::join(ints_coro(10), ", ")); // ok
fmt::print("{}", ints_coro(10) | ranges::to<std::vector>); // ok, but
      expensive
fmt::print("{}", views::iota(0, 10)); // ok, but harder to implement
```

§ 2.5 Specifying formatters for ranges

It's quite important that a `std::string` whose value is `"hello"` gets printed as `hello` rather than something like `[h, e, l, l, o]`.

This would basically fall out no matter how we approach implementing such a thing, so in of itself is not much of a concern. However, for users who have either custom containers or want to customize formatting of a standard container for their own types, they need to make sure that they can provide a specialization which is more constrained than the standard library's for ranges. To ensure that they can do that, I think we need to be clear about the specific constraint we use when we specify this.

§ 2.6 `format` or `std::cout`?

Just `format` is sufficient.

§ 2.7 What about `vector<bool>`?

Nobody expected this section.

The `value_type` of this range is `bool`, which is formattable. But the `reference` type of this range is `vector<bool>::reference`, which is not. In order to make the whole type formattable, we can either make `vector<bool>::reference` formattable (and thus, in general, a range is formattable if both its value and reference types are formattable) or allow formatting to fall back to constructing a `value` for each `reference` (and thus, in general, a range is formattable if at least its value is).

For most ranges, the `value_type` is `remove_cvref_t<reference>`, so there's no distinction here between the two options. And even for `zip`, there's still not much distinction since it just wraps this question in `tuple` since again for most ranges the types will be something like `tuple<T, U>` vs `tuple<T&, U const&>`, so again there isn't much distinction.

`vector<bool>` is one of the very few ranges in which the two types are truly quite different. So it doesn't offer much in the way of a good example here, since `bool` is cheaply constructible from `vector<bool>::reference`. Though it's also very cheap to provide a formatter specialization for `vector<bool>::reference`. We might as well.

3 Proposal

The standard library should add specializations of `formatter` for:

- any type that satisifies `range` whose `value_type` and `reference` are formattable,
- `pair<T, U>` if `T` and `U` are formattable,
- `tuple<Ts...>` if all of `Ts...` are formattable,
- `vector<bool>::reference` (which does as `bool` does).

The choice of formatting is implementation defined (though implementors are encouraged to format ranges and tuples differently).

The standard library should also add a utility `std::format_join` (or any other suitable name, knowing that `std::views::join` already exists), following in the footsteps of `fmt::join`, which allows the user to provide more customization in how ranges and tuples get formatted.

For types like `std::generator<T>` (which are move-only, non-const-iterable ranges), users will have to use `std::format_join` facility.

4 References

[LWG3478] Barry Revzin. views::split drops trailing empty range.
https://wg21.link/lwg3478

[P2168R0] Corentin Jabot, Lewis Baker. 2020-05-16. generator: A Synchronous Coroutine Generator Compatible With Ranges.
https://wg21.link/p2168r0

[P2214R0] Barry Revzin, Conor Hoekstra, Tim Song. 2020-10-15. A Plan for C++23 Ranges.
https://wg21.link/p2214r0